DAX (Data Analysis Expressions) in Power BI

1. Introduction to DAX

DAX, which stands for **Data Analysis Expressions**, is a powerful formula expression language used across Microsoft analytical tools including **Power BI**, **Analysis Services Tabular Models**, and **Power Pivot in Excel**. DAX enables the creation of advanced calculations, analysis, and queries on data stored in tabular data models. It extends the analytical capabilities of Power BI far beyond simple aggregations by providing functions, operators, and syntax for building sophisticated business calculations.

DAX is essential for tasks such as:

- Defining **calculated columns**
- Creating **explicit measures**
- Generating **calculated tables**
- Building **advanced aggregations**
- Implementing **row-level security**
- Executing **DAX queries** in tools like DAX Studio or SQL Server Management Studio
- Performing **context-aware calculations** (filter context, row context, evaluation context)

YATICORP
EMPOWERING EDUCATION



While DAX formulas may resemble Excel formulas, they are more powerful and operate over entire tables and relationships in a data model. Understanding DAX is critical for anyone who wants to build dynamic, scalable, and interactive dashboards in Power BI.

---

2. Key Components of DAX

DAX formulas make use of three main components:

1. **Functions** – SUM, CALCULATE, FILTER, RELATED, SUMX, etc.
2. **Operators** – arithmetic operators (+, -, *, /), logical operators (&&, ||), comparison (=, <>, >).
3. **Values/Fields** – columns, tables, and constants.

These components come together to build the expressions that define how data is aggregated, filtered, or transformed.

---

3. Why DAX is Important

Without DAX, Power BI would be limited to:

- Basic summarizations
- Simple visualizations
- Limited business logic

YATICORP
EMPOWERING EDUCATION

DAX enables deeper analytical features such as:

- Creating KPIs
- Time intelligence (YTD, QTD, YoY changes)
- Custom aggregations
- Financial calculations
- What-if analysis
- Segmentation and cohort analysis

Thus, DAX is the **calculation engine** behind modern Power BI reporting.

---

4. Types of DAX Calculations

DAX calculations broadly fall into **four** categories:

4.1. Measures

Measures are dynamic calculations whose results change depending on:

- The filter context of the report
- The selections made in slicers
- The grouping applied in visuals

Measures **do not create physical columns** in the model; instead, they compute values on the fly as users interact with the report.

Measures come in two types:

Implicit Measures

YATICORP
EMPOWERING EDUCATION

- Automatically created by Power BI
- Triggered when a user drags a numeric field into a visual
- Typical aggregations: **Sum, Count, Average, Min, Max**
- Quick and easy but not customizable
- Limited to basic aggregations

Example:
Dragging *Revenue* field onto a card visual → Power BI automatically shows **Sum of Revenue**.

Explicit Measures

- Created manually by the report developer
- Use DAX formulas in the formula bar
- Offer greater control and flexibility
- Can reference other measures
- Support complex business logic

Example:

```
Total Revenue = SUM(Sales[Revenue])
```

Explicit measures are essential when business rules or custom calculations are required.

4.2. Calculated Columns

Calculated columns:

- Create new columns using DAX expressions
- Are computed row-by-row when data is loaded or refreshed
- Become part of the in-memory model
- Do **not** change based on filter context
- Are useful for data categorization, grouping, or enriching source tables

YATICORP
EMPOWERING EDUCATION

Examples of use cases:

- Conditional classifications (High / Medium / Low)
- Extracting year from date
- Creating unique keys
- Data cleansing and transformations
- Creating flags such as "New Customer" or "Loyal Customer"

Example DAX:

```
Profit Category = IF(Products[Profit Per Unit] > 50, "High Profit", "Low Profit")
```

4.3. Calculated Tables

A calculated table:

- Is a table generated from a DAX expression
- Does not come directly from the data source
- Can combine, filter, or summarize data from other tables
- Refreshes when the model refreshes
- Supports relationships with other tables

Common use cases:

- Creating summary tables
- Creating calendar tables
- Performing what-if analysis
- Creating filtered versions of a table

YATICORP
EMPOWERING EDUCATION

Example:

```
HighProfitProducts = FILTER(Products, Products[Profit Per Unit] > 50)
```

4.4. DAX Queries

DAX queries are written to:

- Explore model data
- Return custom table results
- Validate DAX logic
- Perform advanced troubleshooting

Tools such as **DAX Studio** and **SQL Server Management Studio (SSMS)** allow running these queries.

DAX queries often use functions like:

- EVALUATE
- SUMMARIZE
- FILTER
- ORDER BY

Example:

```
EVALUATE
SUMMARIZE(
    Sales,
    Sales[CustomerID],
    "Total Revenue", SUM(Sales[Revenue])
)
```

Compared to MDX (Multidimensional Expressions), DAX queries are often easier to write and more efficient in tabular models.

YATICORP
EMPOWERING EDUCATION

---

5. Understanding Context in DAX

One of the most important concepts in DAX is **context**:

5.1. Row Context

- Exists in calculated columns
- The expression is evaluated row by row
- Similar to how Excel calculates formulas for each row
- Functions like EARLIER and X functions rely heavily on row context

5.2. Filter Context

- Is created by slicers, filters, report pages
- Determines what subset of data the measure is operating on

- Measures always work based on filter context

Example:

Cases where filter context changes:

- Select a product category → Total Revenue changes
- Select a time period → Calculations adjust accordingly

5.3. Evaluation Context

- Combination of row context and filter context
- Managed internally by DAX when using CALCULATE and iterator functions

CALCULATE is the most important function in DAX because it modifies filter context.

YATICORP
EMPOWERING EDUCATION

6. Connecting Data and Preparing the Model

Before DAX can be used, the data model must be properly set up.

6.1. Importing Data

In the example presented, three data sources were connected:

1. **Customer Data**
2. **Product Data**
3. **Sales Data**

All three came from Excel files with single sheets.

6.2. Using Power Query for Data Preparation

Power Query was used to:

- Promote first row to headers
- Verify data types

- Rename tables
- Fix incorrectly assigned data types

Example correction:
OrderID was wrongly detected as a number. It was converted to **Text** because identifiers should not be aggregated.

6.3. Close & Apply

After cleaning:

- The queries were loaded into Power BI Desktop
- Tables appeared in the Data and Model views

---

7. Identifying Implicit Measures

YATICORP
EMPOWERING EDUCATION

In the Fields pane:

- Numeric fields show a **Σ (sigma)** icon
- These are eligible for implicit measures

Example:

- Quantity → numeric
- Profit → numeric
- Revenue → numeric

Dragging any of these into a visual creates an implicit sum.

Example:

- Adding **Quantity** to a card → Sum of Quantity is shown
- Adding **Revenue** → Sum of Revenue appears

Implicit measures are limited and cannot be reused in more complex DAX calculations.

8. Creating Explicit Measures

Explicit measures are created through the **New Measure** option.

Steps to Create a Measure

1. Select the target table (e.g., Sales).
2. Go to **Table Tools → New Measure**.
3. A formula bar appears above the canvas.
4. Write the DAX expression.
5. Press Enter or click the checkmark.

Example: Creating Total Revenue Measure

Instead of using the implicit sum, a custom explicit measure is created:

```
Total Revenue = SUM(Sales[Revenue])
```

YATICORP
EMPOWERING EDUCATION

Once the measure is added:

- It appears in the Fields list under the Sales table
- It can be used in visuals (card, table, KPI, etc.)
- It becomes reusable in further calculations

Error Handling in Measures

If a measure name conflicts with an existing field (like naming a measure "Revenue"), Power BI alerts:

> *"Name already used. Choose a different name."*

Thus, the measure was named **Total Revenue**.

Displaying the Measure

When added to a card, the measure displays the total revenue computed using DAX.

---

9. Difference: Implicit vs Explicit Measures

Aspect	Implicit Measure	Explicit Measure
Created	Automatically	Manually using DAX
Reusable	No	Yes
Custom logic	Limited	Highly customizable
Appears in Fields	No	Yes
Context handling	Basic	Advanced
Best practice	Not recommended for complex models	Highly recommended

Explicit measures are preferred because they ensure consistency, transparency, and scalability in a Power BI model.

---

10. When to Use What?

YATICORP
EMPOWERING EDUCATION

Use Implicit Measures When:

- You need quick totals
- You are drafting a report
- You are exploring data in the beginning

Use Explicit Measures When:

- You need to perform business logic
- The same calculation is required in multiple visuals
- You need time-intelligence calculations
- You need context-aware formulas
- A calculation depends on other measures

---

11. Practical Examples of DAX Measures

Here are common real-world measures used in dashboards:

Total Sales

```
Total Sales = SUM(Sales[Revenue])
```

Total Quantity

```
Total Quantity = SUM(Sales[Quantity])
```

Average Revenue Per Order

```
Avg Revenue per Order = AVERAGE(Sales[Revenue])
```

Profit Margin

```
Profit Margin = DIVIDE(SUM(Sales[Profit]), SUM(Sales[Revenue]))
```

Year-to-Date Revenue

```
YTD Revenue = TOTALYTD([Total Revenue], Sales[OrderDate])
```

YATICORP
EMPOWERING EDUCATION

---

12. Best Practices for DAX

1. **Use Explicit Measures Always**
   Avoid implicit measures for production reports.

2. **Name Measures Clearly**
   Example: *Total Revenue, Total Profit, YTD Sales*.

3. **Avoid Using Columns Inside Visuals Without Measures**
   Always wrap numeric columns inside measures.

4. **Build a Measure Table**
   Helps organize measures better.

5. **Limit Calculated Columns**
   Use when row-level logic is necessary; otherwise prefer measures.

6. **Use CALCULATE for Advanced Logic**
   It allows modifying filter context and is the most powerful DAX function.

7. **Use Variables (VAR)**
   Improves readability and performance.

---

13. Summary

DAX is the core of Power BI analytics. Understanding how to create measures, calculated columns, and calculated tables is essential for building insightful and interactive dashboards.

In this session, we learned:

- What DAX is and why it is used
- Types of DAX calculations (Measures, Columns, Tables, Queries)
- Difference between implicit and explicit measures
- How to create explicit measures using DAX expressions
- How to prepare data and fix data types before applying DAX
- Practical examples and best practices

YATICORP
EMPOWERING EDUCATION

DAX unlocks the real analytical power of Power BI. With a strong foundation in DAX, users can create deep, meaningful business insights that adapt dynamically to report interactions.